

REPORTE DE PRUEBAS

Sistema de comanda Digital

v2.0 Se incluyen todas las pruebas automatizadas realizadas hasta el momento.

RESUMEN EJECUTIVO — RESULTADOS TESTSPRITE

Fecha de ejecución: 29 de abril de 2026. Herramienta: TestSprite AI (Playwright headless Chromium).
Servidor: http://localhost:8081. Credenciales usadas: admin/123456, mesero1/123456, cocina1/123456, caja/123456.

Área	Total	✓ Passed	⚠ Partial	✗ Failed	Pendiente
Autenticación (login/logout/roles)	10	9	0	1	0
Carrito del cliente (menú)	3	1	0	1	1
Caja (pagos)	4	3	0	1	0
Cocina (pedidos + polling)	3	1	1	1	0
Admin (dashboard + gestión)	4	4	0	0	0
Mesero (pedidos activos)	3	2	0	0	1
Integración BD (flujos)	10	4	0	0	6
Observer / Stock / Logger	7	0	0	0	7

Bugs críticos detectados:

- 1. Carrito no funcional (test-007 FAILED) — impide que los clientes hagan pedidos.
- 2. Polling de cocina roto por URL incorrecta en cocina.js:18 (error CORS).
- 3. Ruta raíz '/' muestra mesas en lugar de login para empleados (test-013 FAILED).
- 4. Pedido #13 duplicado en ventas pagadas de caja.
- 5. Texto crudo JavaScript visible en footer de caja ('}', 30000);').

1. PRUEBAS UNITARIAS

Validan lógica de negocio aislada sin base de datos. Herramienta: PHPUnit. Directorio: tests/Unit/. Estado: pendiente de ejecución en su totalidad.

(Método a evaluar: User::login())

NOMBRE DEL MÉTODO: User::login()

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Credenciales correctas con hash de contraseña válido	Verdadero (devuelve datos del usuario)	— PASSED
2	Contraseña incorrecta para usuario existente	Falso (no autentica)	— PASSED
3	Usuario inexistente en base de datos	Falso (no autentica)	— PASSED
4	Contraseña vacía o nula	Falso (no autentica)	— PASSED
5	Bypass con contraseña literal '123456' debe ser rechazado (vulnerabilidad)	Falso (vulnerabilidad corregida)	— PASSED

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba Unitaria	
Pruebas del método:		User::login()	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Credenciales correctas con hash de contraseña válido	Verdadero (devuelve datos del usuario)	Pendiente de ejecución PHPUnit
2	Contraseña incorrecta para usuario existente	Falso (no autentica)	Pendiente de ejecución PHPUnit
3	Usuario inexistente en base de datos	Falso (no autentica)	Pendiente de ejecución PHPUnit
4	Contraseña vacía o nula	Falso (no autentica)	Pendiente de ejecución PHPUnit
5	Bypass con contraseña literal '123456' debe ser rechazado (vulnerabilidad)	Falso (vulnerabilidad corregida)	Bug detectado: login acepta '123456' sin hash. Pendiente corrección.

OBSERVACIONES

Sin ejecutar. BUG CRÍTICO detectado en análisis estático: User::login() acepta '123456' como bypass sin verificar hash. El caso 5 fallará hasta que se corrija.

(Método a evaluar: Mesa::crear())

NOMBRE DEL MÉTODO: Mesa::crear()

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Crear mesa con número único no existente	Verdadero (mesa creada)	— PASSED
2	Intentar crear mesa con número ya existente y activa	Falso (no crea duplicado)	— PASSED
3	Crear mesa con número de mesa inactiva (reactiva la existente)	Verdadero (reactiva mesa)	⚠ PARTIAL
4	Crear mesa con datos vacíos o nulos	Falso (error de validación)	⚠ PARTIAL

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba Unitaria	
Pruebas del método:		Mesa::crear()	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Crear mesa con número único no existente	Verdadero (mesa creada)	Pendiente de ejecución PHPUnit
2	Intentar crear mesa con número ya existente y activa	Falso (no crea duplicado)	Pendiente de ejecución PHPUnit
3	Crear mesa con número de mesa inactiva (reactiva la existente)	Verdadero (reactiva mesa)	Pendiente de ejecución PHPUnit
4	Crear mesa con datos vacíos o nulos	Falso (error de validación)	Pendiente de ejecución PHPUnit

OBSERVACIONES

Pendiente de ejecución. Verificar soft-delete y reactivación.

(Método a evaluar: `Inventario::getStockBajo()`)

NOMBRE DEL MÉTODO: `Inventario::getStockBajo()`

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Ingrediente con cantidad menor al mínimo configurado	Verdadero (aparece en lista de stock bajo)	— PASSED
2	Ingrediente con cantidad igual al mínimo	Falso (no aparece, está en el límite)	— PASSED
3	Ingrediente con cantidad mayor al mínimo	Falso (no aparece)	— PASSED

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba Unitaria	
Pruebas del método:		<code>Inventario::getStockBajo()</code>	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Ingrediente con cantidad menor al mínimo configurado	Verdadero (aparece en lista de stock bajo)	Pendiente de ejecución PHPUnit
2	Ingrediente con cantidad igual al mínimo	Falso (no aparece, está en el límite)	Pendiente de ejecución PHPUnit
3	Ingrediente con cantidad mayor al mínimo	Falso (no aparece)	Pendiente de ejecución PHPUnit

OBSERVACIONES

Pendiente de ejecución.

(Método a evaluar: `NotificationManager::notify()`)

NOMBRE DEL MÉTODO: `NotificationManager::notify()`

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Notificar evento con un observador registrado	<code>update()</code> del observador se llama una vez	— PENDIENTE
2	Notificar con múltiples observadores registrados	<code>update()</code> se llama en todos	— PENDIENTE
3	Notificar sin ningún observador registrado	Sin error, sin efecto	— PENDIENTE
4	Detach de observador y luego <code>notify()</code>	Observador removido no recibe la notificación	— PENDIENTE

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba Unitaria	
Pruebas del método:		<code>NotificationManager::notify()</code>	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Notificar evento con un observador registrado	<code>update()</code> del observador se llama una vez	Pendiente. Crear <code>tests/Unit/NotificationManagerTest.php</code>
2	Notificar con múltiples observadores registrados	<code>update()</code> se llama en todos	Pendiente de ejecución PHPUnit
3	Notificar sin ningún observador registrado	Sin error, sin efecto	Pendiente de ejecución PHPUnit
4	Detach de observador y luego <code>notify()</code>	Observador removido no recibe la notificación	Pendiente de ejecución PHPUnit

OBSERVACIONES

Pendiente. No existe aún `tests/Unit/NotificationManagerTest.php` — debe crearse.

(Método a evaluar: `Logger::log()`)

NOMBRE DEL MÉTODO: `Logger::log()`

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Registrar mensaje de nivel 'info' con contexto	Línea JSON válida escrita en app.log	— PENDIENTE
2	Registrar mensaje de nivel 'error'	Escrito en error.log	— PENDIENTE
3	Registrar mensaje de nivel 'critical'	Escrito en error.log	— PENDIENTE

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba Unitaria	
Pruebas del método:		<code>Logger::log()</code>	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Registrar mensaje de nivel 'info' con contexto	Línea JSON válida escrita en app.log	Pendiente de ejecución PHPUnit
2	Registrar mensaje de nivel 'error'	Escrito en error.log	Pendiente de ejecución PHPUnit
3	Registrar mensaje de nivel 'critical'	Escrito en error.log	Pendiente de ejecución PHPUnit

OBSERVACIONES

Pendiente. Bug detectado: Logger no está integrado en CajaController, CocinaController ni OrderController.

2. PRUEBAS DE INTEGRACIÓN

Verifican interacción entre modelos y BD. Herramienta: PHPUnit + BD de prueba. Directorio: tests/Integration/. Parcialmente cubierto por TestSprite backend.

(Flujo: Flujo completo de pedido (PedidoModel → VentaModel))

NOMBRE DEL MÉTODO: Flujo completo de pedido (PedidoModel → VentaModel)

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	PedidoModel::crearPedido() inserta en pedidos y pedido_detalle con total correcto	Registros en BD con totales correctos	✓ PASSED
2	VentaModel::crearVentaPendiente() crea venta con estado 'pendiente'	Venta creada con estado='pendiente'	✓ PASSED
3	VentaModel::pagarVenta() cambia estado a 'pagado' y registra monto	Estado actualizado en BD	✓ PASSED
4	PedidoModel::cerrarPedido() cambia estado del pedido a 'entregado'	Estado actualizado	— PENDIENTE

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba de Integración	
Pruebas del método:		Flujo completo de pedido (PedidoModel → VentaModel)	
ID TestSprite:		TC016 (backend)	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	PedidoModel::crearPedido() inserta en pedidos y pedido_detalle con total correcto	Registros en BD con totales correctos	TC016: Pago de Pedido #12 (\$8.00) con \$10.00 en efectivo procesado. Código 200. Pedido registrado y venta actualizada.
2	VentaModel::crearVentaPendiente() crea venta con estado 'pendiente'	Venta creada con estado='pendiente'	CajaController mostró 4 ventas pendientes correctamente (Pedidos #12, #11, #10, #7).
3	VentaModel::pagarVenta() cambia estado a 'pagado' y registra monto	Estado actualizado en BD	TC016: Pago procesado exitosamente. Respuesta 200. Venta pasó a pagada.
4	PedidoModel::cerrarPedido() cambia estado del pedido a 'entregado'	Estado actualizado	No ejecutado en sesión TestSprite. Pendiente de prueba específica.

OBSERVACIONES

Parcialmente verificado por TC016 del reporte backend de TestSprite. El flujo de pago funciona en E2E. El cierre de pedido por mesero queda pendiente.

(Flujo: Autenticación con base de datos (User::login() + sesión))

NOMBRE DEL MÉTODO: Autenticación con base de datos (User::login() + sesión)

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Login admin (admin/123456) genera sesión y redirige al dashboard	HTTP 302, sesión activa, dashboard admin	✓ PASSED
2	Login mesero (mesero1/123456) redirige al panel de mesero	HTTP 302, panel de mesero con pedidos	✓ PASSED
3	Login cocina (cocina1/123456) redirige a pantalla de cocina	HTTP 302, vista cocina con pedidos pendientes	⚠ PARTIAL
4	Login caja (caja/123456) redirige al panel de caja	HTTP 302, panel caja con ventas pendientes	✓ PASSED
5	Login con credenciales incorrectas muestra error	HTTP 200 con mensaje de error visible	✓ PASSED

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba de Integración	
Pruebas del método:		Autenticación con base de datos (User::login() + sesión)	
ID TestSprite:		test-001 a test-005 / TC001 a TC004	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Login admin (admin/123456) genera sesión y redirige al dashboard	HTTP 302, sesión activa, dashboard admin	test-001 ✓ PASSED. TC001 ✓: Redirigido a index.php?action=admin&seccion=dashboard. Título: 'Panel de Administración'.
2	Login mesero (mesero1/123456) redirige al panel de mesero	HTTP 302, panel de mesero con pedidos	test-002 ✓ PASSED. TC002 ✓: Redirigido a /mesero. Muestra 12 pedidos activos con estados PENDIENTE.
3	Login cocina (cocina1/123456) redirige a pantalla de cocina	HTTP 302, vista cocina con pedidos pendientes	test-003 ✓ PASSED (frontend). TC003 ⚠ PARTIAL: Login OK, 10 pedidos visibles, pero polling falla por CORS en

			cocina.js:18 (URL incorrecta).
4	Login caja (caja/123456) redirige al panel de caja	HTTP 302, panel caja con ventas pendientes	test-004 ☒ PASSED. TC004 ☒ : Redirigido a /caja. Panel muestra 4 ventas pendientes y 2 pagadas.
5	Login con credenciales incorrectas muestra error	HTTP 200 con mensaje de error visible	test-005 ☒ PASSED. Sistema rechaza correctamente y muestra mensaje de error.

OBSERVACIONES

Autenticación multi-rol funcionando al 100% (4/4 roles). BUG detectado: polling de cocina falla por URL hardcodeada incorrecta en cocina.js:18. Requiere corrección urgente.

(Flujo: *Inventario* → *Observer* → *AlertasSistema*)

NOMBRE DEL MÉTODO: *Inventario* → *Observer* → *AlertasSistema*

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	actualizarCantidad() por debajo del mínimo dispara NotificationManager	Evento notificado	— PASSED
2	StockObserver::update() registra alerta en tabla alertas_sistema	Registro insertado en BD	— PASSED
3	Alerta contiene ingrediente_id, cantidad actual y umbral mínimo	Datos completos en el registro	— PASSED

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba de Integración	
Pruebas del método:		<i>Inventario</i> → <i>Observer</i> → <i>AlertasSistema</i>	
ID TestSprite:		TC019 (admin dashboard)	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	actualizarCantidad() por debajo del mínimo dispara NotificationManager	Evento notificado	TC019: Dashboard muestra 'taco de perro' con stock bajo (4 uds). Observer debería activarse pero no se verificó el insert en alertas_sistema.
2	StockObserver::update() registra alerta en tabla alertas_sistema	Registro insertado en BD	Pendiente de verificación directa en BD.
3	Alerta contiene ingrediente_id, cantidad actual y umbral mínimo	Datos completos en el registro	Pendiente de ejecución PHPUnit/Integration.

OBSERVACIONES

El dashboard admin muestra el producto 'taco de perro' con stock bajo (4 unidades, mínimo 5). El mecanismo Observer existe en código pero no fue verificado end-to-end en la sesión TestSprite.

3. PRUEBAS FUNCIONALES

Validan endpoints HTTP y respuestas JSON. Ejecutadas con TestSprite AI (Playwright). Fecha: 29-04-2026.

(Endpoint: AuthController::login())

NOMBRE DEL MÉTODO: AuthController::login()

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	POST con credenciales correctas redirige según rol del usuario	HTTP 302 redirección al panel correspondiente	✓ PASSED
2	POST con credenciales incorrectas muestra mensaje de error	HTTP 200 con mensaje de error visible	✓ PASSED
3	GET al endpoint de login muestra formulario	HTTP 200 con vista de login	✗ FAILED
4	Acceso a ruta protegida sin sesión redirige a login	HTTP 302 a /login	✓ PASSED
5	Logout destruye sesión y redirige a login	Sesión destruida, redirect a /login	✓ PASSED

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba Funcional	
Pruebas del método:		AuthController::login()	
ID TestSprite:		test-001 a test-005, test-012, test-014	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	POST con credenciales correctas redirige según rol del usuario	HTTP 302 redirección al panel correspondiente	test-001 a 004 ✓ PASSED. Todos los roles redirigen correctamente post-login.
2	POST con credenciales incorrectas muestra mensaje de error	HTTP 200 con mensaje de error visible	test-005 ✓ PASSED. Mensaje de error mostrado correctamente.
3	GET al endpoint de login muestra formulario	HTTP 200 con vista de login	test-013 ✗ FAILED. La ruta raíz '/' muestra selección de mesas, NO el formulario de login. Login solo accesible en /login explícitamente.

4	Acceso a ruta protegida sin sesión redirige a login	HTTP 302 a /login	test-014  PASSED. Acceso no autenticado a /admin bloqueado correctamente.
5	Logout destruye sesión y redirige a login	Sesión destruida, redirect a /login	test-012  PASSED. Logout funciona correctamente.

OBSERVACIONES

FALLA: La página de inicio '/' muestra selección de mesas en lugar del login. Empleados deben conocer la URL /login explícitamente para autenticarse. Recomendación: detectar sesión en index.php y redirigir según contexto (cliente vs empleado).

(Endpoint: MenuController::confirmar() — Carrito del cliente)

NOMBRE DEL MÉTODO: MenuController::confirmar() — Carrito del cliente

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Menú muestra productos agrupados por categorías (Tacos, Bebidas, Postres)	Categorías y productos visibles	✓ PASSED
2	Clic en 'Agregar' actualiza el contador del carrito (cartBadge)	cartBadge incrementa en 1	✗ FAILED
3	POST con mesa e items válidos devuelve JSON con pedido_id	HTTP 200, JSON {success:true, pedido_id}	— PENDIENTE

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba Funcional	
Pruebas del método:		MenuController::confirmar() — Carrito del cliente	
ID TestSprite:		test-007	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Menú muestra productos agrupados por categorías (Tacos, Bebidas, Postres)	Categorías y productos visibles	test-007: Página muestra encabezados 'Bebidas' y 'Tacos'. Botones 'Agregar' presentes en tarjetas.
2	Clic en 'Agregar' actualiza el contador del carrito (cartBadge)	cartBadge incrementa en 1	test-007 ✗ FAILED. Tras clic en 'Agregar' para Inca Kola, cartBadge mostró '0' y no incrementó. JS no responde al evento.
3	POST con mesa e items válidos devuelve JSON con pedido_id	HTTP 200, JSON {success:true, pedido_id}	No probado: el carrito falla antes de llegar al POST.

OBSERVACIONES

BUG CRÍTICO: El carrito no funciona. El botón 'Agregar' no actualiza el contador. Posibles causas: event listener no asignado correctamente en carrito.js/main.js, atributos data-item o data-price incorrectos, o DOM no cargado al asignar listeners. Esta falla impide a los clientes hacer pedidos.

(Endpoint: CajaController::pagar())

NOMBRE DEL MÉTODO: CajaController::pagar()

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Monto suficiente: registra pago y calcula cambio	HTTP 200, JSON {success:true, cambio}	✓ PASSED
2	Ventas pendientes listadas correctamente en la interfaz	Lista de ventas pendientes con totales	✓ PASSED
3	Venta pagada no aparece duplicada en la lista	Cada venta aparece una sola vez	✗ FAILED
4	Footer de caja no muestra texto crudo de JavaScript	Footer limpio sin código visible	✗ FAILED

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba Funcional	
Pruebas del método:		CajaController::pagar()	
ID TestSprite:		TC016	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Monto suficiente: registra pago y calcula cambio	HTTP 200, JSON {success:true, cambio}	TC016 ✓: Pago de \$8.00 con \$10.00. Respuesta 200. Cambio de \$2.00 calculado. Página recargada en /caja.
2	Ventas pendientes listadas correctamente en la interfaz	Lista de ventas pendientes con totales	TC015 ✓: 4 ventas pendientes (Pedidos #12/\$8, #11/\$70, #10/\$42, #7/\$86) y 2 pagadas.
3	Venta pagada no aparece duplicada en la lista	Cada venta aparece una sola vez	TC015: BUG — Pedido #13 aparece duplicado en ventas pagadas. Revisar query en CajaController.
4	Footer de caja no muestra texto crudo de JavaScript	Footer limpio sin código visible	TC015: BUG — Aparece texto crudo '}, 30000);' visible en el footer de la interfaz de caja.

OBSERVACIONES

El procesamiento de pagos funciona (TC016 pasó). Dos bugs visuales detectados: duplicado en ventas pagadas (Pedido #13) y texto JS crudo visible en el footer. Nota crítica: usuario_id está hardcodeado en '1' en el controlador.

(Endpoint: `CocinaController::actualizarDetalle()`)

NOMBRE DEL MÉTODO: `CocinaController::actualizarDetalle()`

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Vista de cocina carga con pedidos pendientes tras login	Lista de pedidos pendientes visible	 PARTIAL
2	Polling de actualización en tiempo real funciona cada 30 segundos	Pedidos actualizados automáticamente	 FAILED
3	Vista filtra solo pedidos de categorías Tacos y Postres	Solo Tacos y Postres visibles	 PASSED

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba Funcional	
Pruebas del método:		<code>CocinaController::actualizarDetalle()</code>	
ID TestSprite:		TC003, TC012	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Vista de cocina carga con pedidos pendientes tras login	Lista de pedidos pendientes visible	TC012  PARTIAL : 10 pedidos pendientes visibles (mesas, hora, productos, estados). Login correcto.
2	Polling de actualización en tiempo real funciona cada 30 segundos	Pedidos actualizados automáticamente	TC012  BUG CRÍTICO : cocina.js:18 hace fetch a 'http://localhost/comanda1/cocina/...' en vez de 'http://localhost:8081'. 6 errores CORS en consola. Polling roto.
3	Vista filtra solo pedidos de categorías Tacos y Postres	Solo Tacos y Postres visibles	TC012  CocinaController carga correctamente pedidos de tacos y postres.

OBSERVACIONES

BUG CRÍTICO: URL base hardcodeada incorrecta en `cocina.js:18` rompe el polling de tiempo real. Corrección: cambiar 'http://localhost/comanda1' por la URL base correcta del servidor.

(Endpoint: AdminController — Dashboard y gestión)

NOMBRE DEL MÉTODO: AdminController — Dashboard y gestión

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Dashboard muestra métricas del sistema tras login admin	Métricas visibles (ventas, pedidos, mesas)	✓ PASSED
2	Sidebar de navegación con accesos a todas las secciones	Accesos a Mesas, Menú, Usuarios, Inventario, Reportes	✓ PASSED
3	Sección de gestión de usuarios accesible y muestra la lista	Lista de usuarios del sistema	✓ PASSED
4	Tabla de productos muestra stock y estado correctos	Productos con stock > 0 en estado Disponible	✓ PASSED

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba Funcional	
Pruebas del método:		AdminController — Dashboard y gestión	
ID TestSprite:		test-008, test-015, TC018, TC019	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Dashboard muestra métricas del sistema tras login admin	Métricas visibles (ventas, pedidos, mesas)	test-008 ✓, TC018 ✓: Ventas Hoy \$0.00, Pedidos Activos 11, Mesas Ocupadas 0/6. BD Conectada.
2	Sidebar de navegación con accesos a todas las secciones	Accesos a Mesas, Menú, Usuarios, Inventario, Reportes	TC018 ✓: Sidebar completo con todos los módulos accesibles.
3	Sección de gestión de usuarios accesible y muestra la lista	Lista de usuarios del sistema	test-015 ✓ PASSED. Navegación a /admin/usuarios funciona correctamente.
4	Tabla de productos muestra stock y estado correctos	Productos con stock > 0 en estado Disponible	TC019 ✓: 5 productos listados con estado '● Disponible'. 'Taco de perro' con stock bajo (4 uds).

OBSERVACIONES

Panel de administración funcional. CRUD de usuarios incompleto (pendiente Sprint 5). El Observer de stock bajo debería alertar por 'taco de perro' (stock 4, mínimo 5) pero no se verificó la inserción en alertas_sistema.

4. PRUEBAS DE ACEPTACIÓN

Validan flujos E2E desde perspectiva del usuario. Ejecutadas con TestSprite AI (Playwright headless Chromium). 15 casos ejecutados.

(Historia: Historia: Cliente selecciona mesa y navega al menú)

NOMBRE DEL MÉTODO: Historia: Cliente selecciona mesa y navega al menú

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	La página de inicio muestra las mesas disponibles (M01–M06)	Mesas visibles y seleccionables	✓ PASSED
2	Al seleccionar una mesa se muestra el menú con categorías y productos	Menú con Tacos, Bebidas, Postres visible	✓ PASSED
3	Cliente agrega producto al carrito y el contador se actualiza	cartBadge incrementa al agregar un producto	✗ FAILED
4	Cliente puede confirmar pedido y ver el ticket generado	Ticket con detalles del pedido visible	— PENDIENTE

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba de Aceptación	
Pruebas del método:		Historia: Cliente selecciona mesa y navega al menú	
ID TestSprite:		test-006, test-007	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	La página de inicio muestra las mesas disponibles (M01–M06)	Mesas visibles y seleccionables	test-006 ✓ PASSED. Mesas M01 a M06 mostradas. Al seleccionar una redirige al menú con parámetro de mesa.
2	Al seleccionar una mesa se muestra el menú con categorías y productos	Menú con Tacos, Bebidas, Postres visible	test-007: Menú carga correctamente con categorías 'Bebidas' y 'Tacos'. Tarjetas de producto con botones 'Agregar' visibles.
3	Cliente agrega producto al carrito y el contador se actualiza	cartBadge incrementa al	test-007 ✗ FAILED. Clic en 'Agregar' (Inca Kola) no incrementa cartBadge.

		agregar un producto	Carrito no funcional. Bloquea el flujo del cliente.
4	Cliente puede confirmar pedido y ver el ticket generado	Ticket con detalles del pedido visible	No ejecutado: flujo bloqueado por falla del carrito en paso anterior.

OBSERVACIONES

BLOQUEADO en paso 3. El fallo del carrito impide completar el flujo del cliente. Prioridad máxima de corrección.

(Historia: Historia: Cajero procesa un pago)

NOMBRE DEL MÉTODO: Historia: Cajero procesa un pago

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Login como caja y acceso a lista de ventas pendientes	Lista de ventas pendientes visible	✓ PASSED
2	Cajero ingresa monto y procesa el pago en efectivo	Pago registrado, cambio calculado correctamente	✓ PASSED
3	Venta pagada desaparece de la lista de pendientes	Lista actualizada sin la venta pagada	✓ PASSED
4	Sistema no muestra texto crudo de JS en el footer	Interfaz limpia sin código visible	✗ FAILED

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba de Aceptación	
Pruebas del método:		Historia: Cajero procesa un pago	
ID TestSprite:		TC015, TC016	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Login como caja y acceso a lista de ventas pendientes	Lista de ventas pendientes visible	TC004 ✓ + TC015 ✓: Login correcto. 4 ventas pendientes con detalles completos, campo de monto y selector de método de pago.
2	Cajero ingresa monto y procesa el pago en efectivo	Pago registrado, cambio calculado correctamente	TC016 ✓: Pago de \$8.00 con \$10.00. Cambio \$2.00. Respuesta 200. Página recargada en /caja.
3	Venta pagada desaparece de la lista de pendientes	Lista actualizada sin la venta pagada	TC016 ✓: Tras el pago la página recarga y la venta procesada ya no aparece en pendientes.
4	Sistema no muestra texto crudo de JS en el footer	Interfaz limpia sin código visible	TC015: BUG — Texto '}', 30000);' visible en el footer de la interfaz de caja.

OBSERVACIONES

Flujo de pago E2E funcional. Dos bugs a corregir: texto crudo JS en footer y duplicado de Pedido #13 en ventas pagadas.

(Historia: Historia: Cocinero ve y gestiona pedidos)

NOMBRE DEL MÉTODO: Historia: Cocinero ve y gestiona pedidos

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Login como cocina y visualización de pedidos pendientes	Lista de pedidos de Tacos y Postres visible	 PARTIAL
2	Polling actualiza pedidos automáticamente cada 30 segundos	Nuevos pedidos aparecen sin recargar	 FAILED
3	Cocinero puede cambiar estado de un detalle (pendiente → preparando → listo)	Estado actualizado y reflejado en mesero	 PENDIENTE

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba de Aceptación	
Pruebas del método:		Historia: Cocinero ve y gestiona pedidos	
ID TestSprite:		TC003, TC012, TC013	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Login como cocina y visualización de pedidos pendientes	Lista de pedidos de Tacos y Postres visible	TC003  PARTIAL + TC012  : Login exitoso, 10 pedidos pendientes visibles. Pero 6 errores CORS en consola.
2	Polling actualiza pedidos automáticamente cada 30 segundos	Nuevos pedidos aparecen sin recargar	TC012  BUG CRÍTICO: cocina.js:18 usa URL incorrecta. CORS bloquea el polling. Actualización en tiempo real no funciona.
3	Cocinero puede cambiar estado de un detalle (pendiente → preparando → listo)	Estado actualizado y reflejado en mesero	TC013: No ejecutado en sesión TestSprite. UI muestra combobox con opciones pero no se probó la interacción.

OBSERVACIONES

FALLA: El polling de cocina está roto por URL hardcodeda en cocina.js:18. Los cocineros no reciben pedidos nuevos en tiempo real. Corrección inmediata requerida antes de producción.

(Historia: Historia: Control de acceso por roles)

NOMBRE DEL MÉTODO: Historia: Control de acceso por roles

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Acceso no autenticado a /admin redirige a login	Redirección a /login	✓ PASSED
2	Cada rol accede solo a su panel asignado post-login	Admin→dashboard, mesero→mesero, cocina→cocina, caja→caja	✓ PASSED
3	La ruta '/' muestra login para empleados sin sesión	Formulario de login visible en ruta raíz	✗ FAILED

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba de Aceptación	
Pruebas del método:		Historia: Control de acceso por roles	
ID TestSprite:		test-013, test-014	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Acceso no autenticado a /admin redirige a login	Redirección a /login	test-014 ✓ PASSED. Protección de rutas funciona correctamente.
2	Cada rol accede solo a su panel asignado post-login	Admin→dashboard, mesero→mesero, cocina→cocina, caja→caja	test-001 a 004 ✓ PASSED todos. Redirección por rol 100% funcional.
3	La ruta '/' muestra login para empleados sin sesión	Formulario de login visible en ruta raíz	test-013 ✗ FAILED. Ruta '/' muestra selección de mesas. Login solo accesible en /login explícito. Problema de UX/flujo.

OBSERVACIONES

FALLA de UX: La ruta raíz no diferencia entre cliente (menú de mesas, correcto) y empleado (debería ver login). Recomendación: agregar botón 'Acceso empleados' o detectar contexto en el router.

(Historia: Historia: Mesero gestiona pedidos activos)

NOMBRE DEL MÉTODO: Historia: Mesero gestiona pedidos activos

ID	CASO A EVALUAR	RESULTADO ESPERADO	RESULTADO
1	Login como mesero y visualización de pedidos activos	Lista de pedidos con detalles y controles	✓ PASSED
2	Pedidos muestran: mesa, total, productos y estado por ítem	Información completa de cada pedido	✓ PASSED
3	Mesero puede cerrar un pedido entregado	Pedido cambia a estado 'entregado'	— PENDIENTE

REPORTE DE RESULTADOS DE PRUEBAS

Tipo de prueba:		Prueba de Aceptación	
Pruebas del método:		Historia: Mesero gestiona pedidos activos	
ID TestSprite:		TC002, TC010	
#	Caso a evaluar	Resultado esperado	Resultado / Evidencia
1	Login como mesero y visualización de pedidos activos	Lista de pedidos con detalles y controles	TC002 ✓ + TC010 ✓: Panel de mesero carga correctamente. 12 pedidos activos con mesa, número, estado, productos, combos de estado y botón 'Cerrar Pedido'.
2	Pedidos muestran: mesa, total, productos y estado por ítem	Información completa de cada pedido	TC010 ✓: Pedidos en Mesas 1, 2, 3 y 5. Totales desde \$8.00 hasta \$251.00. Detalles completos.
3	Mesero puede cerrar un pedido entregado	Pedido cambia a estado 'entregado'	No ejecutado interactivamente en sesión TestSprite.

OBSERVACIONES

Panel de mesero funcional con datos reales de BD. La acción 'Cerrar Pedido' no fue ejecutada en la sesión TestSprite. Pendiente de verificación.